VoiceOps 2026

Capstone Project Report · ZentroMart AI Customer Support Agent

LLM & Generative AI Course · Consulting & Analytics Club, IIT Guwahati × Gnani.ai

Executive summary

This project package implements a starter conversational customer-support agent for ZentroMart, a fictional Indian online store. The intended system answers policy questions from company documents, keeps conversational context, uses the supplied synthetic order lookup and ticket tools, and adds voice input/output through Gnani.ai speech models. The implementation prioritizes policy grounding, verification before order disclosure, honest escalation status, and reproducible setup.

1. Problem statement and objectives

Customer support teams repeatedly answer questions about late deliveries, cancellations, returns, refunds, product warranties, and membership. A useful agent must do more than produce fluent text: it must retrieve the applicable company policy, avoid inventing facts, ask clarifying questions when details are missing, and preserve context across follow-up turns. When customer-specific information is requested, it must verify the caller before revealing order details. When a human decision is required, it must create a ticket and only claim success after the tool confirms creation.

The capstone brief calls for four stages: (1) prompting-only LLM baseline, (2) retrieval-augmented generation (RAG), (3) multi-turn conversations and tools, and (4) voice integration. It specifies Gnani.ai as the required provider for both speech-to-text (STT) and text-to-speech (TTS).

2. Architecture and data flow

Voice input → **Gnani STT** → **conversational agent** → **retrieval / LLM / mock tools** → **Gnani TTS** → **spoken reply.** The text agent can also be run independently for debugging. Knowledge-base Markdown files are split by section headings that contain section IDs. Each chunk retains its source filename, title, status, and effective date. A lexical ranker scores overlap between query tokens and chunk tokens, boosts current documents, and heavily penalizes superseded policies. Retrieved chunks are supplied to the language model with a system instruction to answer only from evidence and verified tool results.

The package includes a baseline lexical retriever rather than a vector database. This choice keeps the starter lightweight and transparent; semantic embeddings and a vector index are a recommended improvement. When no LLM API key is configured, the fallback returns a relevant policy excerpt instead of fabricating a synthesized policy answer.

3. Conversation, tools, and safeguards

The system prompt instructs the assistant to identify itself as an AI assistant, answer concisely, maintain context, and say when the evidence is insufficient. Recent user and assistant messages are retained for follow-up questions. The reference date is 20 October 2026, as specified in the brief. Business-day calculations must use Monday–Saturday while excluding the supplied holiday dates; return windows are calendar days.

Order privacy. The supplied lookup tool does not return order status or items until a caller provides a matching registered phone suffix (last four digits) or delivery pincode. The agent asks for an order ID first, then asks for a verification value. It must not reveal expected values or disclose details on a failed check. The supplied mock tool is explicitly educational and does not provide production-grade authentication.

Human handover. The supplied create_ticket tool requires a sufficiently detailed summary and returns a created flag. The agent only confirms ticket creation when created=True; otherwise it tells the customer that the handover could not be confirmed. The summary should capture the issue, checks already made, and reason for escalation.

Voice. The wrapper uses Gnani's Python SDK clients for STT and TTS, with configurable language and voice/model environment variables. The brief's example uses en-IN and the timbre-v2.5 model. Each participant must use their own Gnani account; keys must be stored in environment variables and never committed to the repository. The SDK's currently installed signatures and accepted audio formats must be verified during testing.

4. Included files

• **src/agent.py** — section parsing, lexical retrieval, grounded LLM call, conversational context, order verification flow, and ticket handover.
• **src/voice.py** and **run_audio.py** — optional batch audio pipeline using Gnani STT/TTS only.
• **tools/tools.py** — supplied synthetic order lookup and ticket creation tools.
• **validate_dataset.py** — supplied integrity checks for row counts, keys, joins, totals, evidence references, and verification behavior.
• **kb/** — five supplied policy documents, including the current returns policy and a superseded policy for precedence testing.
• **practice/** — supplied practice question and conversation CSVs.
• **README.md**, **requirements.txt**, **.env.example**, and **evaluate_smoke.py** — setup, configuration template, and a small smoke test.

5. Evaluation plan and current status

The brief specifies 52 single-question cases and 23 conversations containing 96 customer turns. It awards marks for grounded answers (25), retrieval quality (15), conversation handling (20), the Step 1 versus Step 2 comparison (10), end-to-end voice operation (10), voice experience (10), and report/demo (10).

Results status: not yet measured. This package does not claim an accuracy score, retrieval score, or successful voice run because the complete official dataset and service credentials were not available for an end-to-end run at packaging time. The uploads contained five policy files and two practice CSVs, but not the full nine CSV data tables and all 21 knowledge-base documents described in the brief. Consequently, the full validator and order tools require the remaining official dataset files.

Before final submission, download the official dataset ZIP and copy the full kb/ and data/ folders into the expected locations. Run `python validate_dataset.py`. Then evaluate a prompting-only baseline and the RAG version on the same 52 questions, record correct/total and failure categories, and process conversations in conversation_id/turn order. Check each expected tool call and gold_evidence reference. Finally, test audio input and spoken output with Gnani, save sample outputs, and record a two-minute demo. Do not enter estimated or invented scores in the report.

6. Limitations and improvement roadmap

The lexical retriever uses word overlap and can miss paraphrases, negation, and policy interactions. Improve it by using embeddings and a vector index, adding a minimum relevance threshold, and logging retrieved section IDs for every response. Add explicit date-aware policy selection, unit tests for return-window boundaries and holiday-aware refund timelines, and structured intent/tool routing instead of relying only on keyword checks. Expand evaluation to include abstention quality, source precision, tool-call accuracy, verification retry limits, and ASR transcription failures.

The supplied files and mock data are for an educational simulation only. The package is a project starter that must be validated against the complete dataset before it can be represented as a tested final implementation.

7. Submission checklist

1. Add all missing official dataset files locally and validate them.
2. Run the agent and official evaluation; replace the pending-results note only with measured results.
3. Test Gnani STT/TTS with your own API key and record a two-minute demo.
4. Push the code to a public GitHub repository without `.env`, API keys, virtual environments, or a dataset copy.
5. Add the public GitHub and demo URLs to the README and submission form.
6. Upload this report PDF and any required ZIP to Unstop, then confirm submission.